

React JS – Day 1 Notes

1. What is React?

React is an open-source JavaScript library used to build **interactive, component-based user interfaces (UI)**.

Key ideas

- Component-based development
 - Reusable UI
 - JSX
 - Declarative rendering
 - State and Props
 - Reconciliation
 - DOM updates
-

2. React Project Setup

Using Vite

Vite is a modern build tool commonly used to create React projects.

```
npm create vite@latest
```

Select:

```
Project Name → my-react-app  
Framework → React  
Variant → JavaScript
```

Then:

```
cd my-react-app  
npm install  
npm run dev
```

Project Flow

```
Create Project
  ↓
Install Dependencies
  ↓
Project Files
  ↓
Components
  ↓
Run Application
```

3. Vite React Project Structure

```
my-react-app/
|
├─ node_modules/
├─ public/
├─ src/
|   ├─ assets/
|   ├─ App.jsx
|   ├─ App.css
|   ├─ index.css
|   └─ main.jsx
├─ index.html
├─ package.json
└─ vite.config.js
```

Beginner Important Files

```
index.html
main.jsx
App.jsx
```

4. index.html

`index.html` is the HTML entry page loaded by the browser.

It contains the root element:

```
<div id="root"></div>
```

React mounts the application inside this element.

Think

```
HTML
↓
Empty #root container
↓
React builds UI inside it
```

5. main.jsx

`main.jsx` is commonly the **entry point** of a Vite React application.

Example:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Important

```
document.getElementById("root")
```

finds:

```
<div id="root"></div>
```

Then:

```
.render(<App />);
```

renders the React application.

Flow

```
index.html
  ↓
#root
  ↓
main.jsx
  ↓
createRoot()
  ↓
<App />
  ↓
Application UI
```

6. App.jsx

`App.jsx` is commonly used as the **root application component**.

It is a convention, not a mandatory React filename.

Example:

```
function App() {
  return (
    <div>
      <h1>Welcome to React</h1>
      <p>My First Application</p>
    </div>
  );
}

export default App;
```

Basic Flow

```
main.jsx
  ↓
App.jsx
  ↓
Other Components
```

7. What is a Component?

A **component** is a reusable piece of UI.

Modern React components are commonly JavaScript functions that return JSX.

Function Component

```
function Welcome() {
  return <h1>Hello Student</h1>;
}

export default Welcome;
```

Arrow Function Component

```
const Welcome = () => {
  return <h1>Hello Student</h1>;
};

export default Welcome;
```

8. Component Naming Rule

Component names should start with a **capital letter**.

Correct:

```
function Header() {  
  return <h1>Header</h1>;  
}
```

Use:

```
<Header />
```

React uses capitalization to distinguish custom components from HTML/host elements.

HTML Elements

```
<div>  
<button>  
<input>
```

React Components

```
<Header />  
<Navbar />  
<Footer />
```

9. Why Components are Important

Components help divide a large application into smaller, manageable pieces.

Example: E-commerce Website

Without components:

```
One Huge File  
├─ Navbar  
├─ Banner  
├─ Products  
├─ Cart  
└─ Footer
```

With components:

```
App
├──
├── Navbar
├── Banner
├── ProductList
│   ├── ProductCard
│   ├── ProductCard
│   └── ProductCard
├── Cart
└── Footer
```

Benefits

- Reusability
- Maintainability
- Readability
- Separation of concerns
- Easier testing
- Better project structure

10. Multiple Components

Recommended structure:

```
src/
├──
├── components/
│   ├── Header.jsx
│   ├── Content.jsx
│   └── Footer.jsx
├──
├── App.jsx
└── main.jsx
```

Header.jsx

```
const Header = () => {
  return (
    <header>
      <h1>My React App</h1>
    </header>
  );
};
```

```
};  
  
export default Header;
```

Content.jsx

```
const Content = () => {  
  return (  
    <main>  
      <h2>Welcome</h2>  
      <p>Learning React is fun!</p>  
    </main>  
  );  
};  
  
export default Content;
```

Footer.jsx

```
const Footer = () => {  
  return (  
    <footer>  
      <p>Copyright 2026</p>  
    </footer>  
  );  
};  
  
export default Footer;
```

11. Importing Components

In `App.jsx`:

```
import Header from "../components/Header";  
import Content from "../components/Content";  
import Footer from "../components/Footer";
```

Then:

```
const App = () => {  
  return (  
    <div>  
      <Header />  
      <Content />  
      <Footer />  
    </div>  
  );  
};  
  
export default App;
```

Component Flow

```
main.jsx  
  ↓  
<App />  
  ↓  
App.jsx  
  ↓  
Header  
Content  
Footer  
  ↓  
Complete UI
```

12. JSX

JSX = JavaScript XML

JSX is a syntax extension commonly used with React that allows HTML-like markup to be written inside JavaScript.

Example:

```
const App = () => {  
  return (  
    <h1>Hello React</h1>  
  );  
};
```

This:

```
<h1>Hello React</h1>
```

is JSX.

Important

Browsers do not directly execute JSX as ordinary JavaScript.

Build tools such as Vite transform JSX into JavaScript that browsers can execute.

13. JavaScript Inside JSX

Use curly braces:

```
{ }
```

Example:

```
const App = () => {  
  
  const name = "Arun";  
  const age = 22;  
  
  return (  
    <div>  
      <h1>Hello {name}</h1>  
      <p>Age: {age}</p>  
    </div>  
  );  
};
```

Output:

```
Hello Arun  
Age: 22
```

Easy Memory

```
JSX
+
{ JavaScript Expression }
```

14. JSX Rules

Rule 1: Single Parent Element

Incorrect:

```
return (
  <h1>Hello</h1>
  <p>Welcome</p>
);
```

Correct:

```
return (
  <div>
    <h1>Hello</h1>
    <p>Welcome</p>
  </div>
);
```

Or use a Fragment:

```
return (
  <>
    <h1>Hello</h1>
    <p>Welcome</p>
  </>
);
```

15. className

In normal HTML:

```
<h1 class="title">
```

In JSX:

```
<h1 className="title">  
  Hello  
</h1>
```

Remember

```
HTML → class  
JSX → className
```

16. Self-Closing Elements

JSX requires elements without children to be self-closed.

Examples:

```
  
  
<input type="text" />  
  
<br />
```

17. Events in React

HTML:

```
<button onClick="showData()">
  Click
</button>
```

React:

```
<button onClick={showData}>
  Click
</button>
```

Example

```
const App = () => {

  const showMessage = () => {
    alert("Hello React");
  };

  return (
    <button onClick={showMessage}>
      Click Me
    </button>
  );
};

export default App;
```

Important

React uses:

```
onClick
```

not:

```
onclick
```

18. Real DOM

DOM = Document Object Model

The browser represents an HTML document as a tree of elements/nodes.

HTML:

```
<div>
  <h1>Hello</h1>
  <p>Welcome</p>
</div>
```

Conceptual DOM:

```
div
├── h1
│   └── Hello
└── p
    └── Welcome
```

The browser's actual DOM is commonly called the **Real DOM**.

19. Traditional DOM Manipulation

Vanilla JavaScript:

```
<h1 id="count">0</h1>

<button id="btn">
  Add
</button>
```

JavaScript:

```
let count = 0;

const heading = document.getElementById("count");
const button = document.getElementById("btn");
```

```
button.addEventListener("click", () => {  
  count++;  
  heading.textContent = count;  
});
```

Here, we manually:

```
Find Element  
  ↓  
Listen for Event  
  ↓  
Change Element
```

20. Virtual DOM

Virtual DOM is a common term for React's in-memory representation of the UI.

When state changes, React determines what the next UI should look like and figures out what changes need to be committed to the browser DOM.

Basic Concept

```
State Change  
  ↓  
React Renders  
  ↓  
New UI Representation  
  ↓  
Reconciliation  
  ↓  
Required DOM Changes  
  ↓  
Browser UI Updated
```

Important Interview Point

Do not say:

"React is fast because Virtual DOM is always faster than the Real DOM."

A better explanation:

React provides a declarative rendering model and determines the necessary UI updates for us. Performance depends on the application and the work being performed.

21. Real DOM vs Virtual DOM

Real DOM	Virtual DOM Concept
Browser's actual DOM	React's in-memory UI representation
Exists in the browser	Managed internally by React
Actual page changes happen here	Helps determine required changes
Browser rendering work occurs here	React calculates/reconciles UI changes

22. Reconciliation

Reconciliation is the process React uses to determine how a new rendered UI relates to the previous UI and what updates are necessary.

Flow

```
graph TD
    A[Previous UI] --> B[State Changes]
    B --> C[New Render]
    C --> D[Reconciliation]
    D --> E[Determine Differences]
    E --> F[Commit Necessary Updates]
    F --> G[Real DOM]
```

23. Reconciliation Example

Initial UI:

```
<h1>Count: 0</h1>
<p>Welcome</p>
```

After update:

```
<h1>Count: 1</h1>
<p>Welcome</p>
```

The paragraph has not logically changed.

The relevant change is:

```
Count: 0
  ↓
Count: 1
```

React determines the required update and commits it to the DOM.

24. State Update

Example using `useState()`:

```
import { useState } from "react";

const App = () => {

  const [count, setCount] = useState(0);

  const increase = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <h1>Count: {count}</h1>

      <button onClick={increase}>
        Add
      </button>
    </div>
  );
};
```

```
};  
  
export default App;
```

Concept

Initial:

```
count = 0
```

Click button:

```
setCount(1)
```

Then:

```
State Changes  
  ↓  
Component Scheduled to Render  
  ↓  
App() Runs Again  
  ↓  
New JSX Result  
  ↓  
Reconciliation  
  ↓  
DOM Commit  
  ↓  
Count: 1
```

25. React Rendering Process

Must Remember

```
USER ACTION  
  ↓  
EVENT  
  ↓  
STATE UPDATE  
  ↓
```

```
RENDER
  ↓
RECONCILIATION
  ↓
COMMIT
  ↓
BROWSER UI UPDATE
```

Example:

```
Click Button
  ↓
onClick
  ↓
setCount()
  ↓
Component renders
  ↓
React reconciles
  ↓
Necessary DOM update
  ↓
New value visible
```

26. React vs Traditional JavaScript

Vanilla JavaScript

```
const title = document.getElementById("title");

title.textContent = "Welcome";
```

We explicitly tell JavaScript:

```
Find this element
  ↓
Change this property
```

React

```
const App = () => {  
  
  const name = "Welcome";  
  
  return (  
    <h1>{name}</h1>  
  );  
};
```

React uses a more **declarative** approach:

```
Data / State  
  ↓  
Describe UI  
  ↓  
React Rendering  
  ↓  
DOM Updates
```

27. return in a Component

A component commonly uses `return` to describe the UI it should produce.

```
const Header = () => {  
  
  return (  
    <h1>Welcome</h1>  
  );  
  
};
```

Think

```
Component Function  
  ↓  
return  
  ↓  
JSX
```

↓
React renders UI

28. export default

Example:

```
const Header = () => {  
  return <h1>Header</h1>;  
};  
  
export default Header;
```

Another file:

```
import Header from "./Header";
```

Use:

```
<Header />
```

Flow

```
Header.jsx  
  ↓  
export default  
  ↓  
App.jsx  
  ↓  
import  
  ↓  
<Header />
```

29. CSS in React

App.css:

```
.title {  
  color: blue;  
  font-size: 40px;  
}
```

App.jsx:

```
import "./App.css";  
  
const App = () => {  
  
  return (  
    <h1 className="title">  
      Hello React  
    </h1>  
  );  
  
};  
  
export default App;
```

30. Images in React

Image from public

```

```

Image from src/assets

```
import profile from "./assets/profile.jpg";  
  
const App = () => {  
  
  return (  
    <img  
      src={profile}  
      alt="Profile"  
    >
```

```
    />  
  );  
};
```

31. Simple React Application

Header.jsx

```
const Header = () => {  
  return (  
    <header>  
      <h1>Student Portal</h1>  
    </header>  
  );  
};  
  
export default Header;
```

Student.jsx

```
const Student = () => {  
  
  const name = "Arun";  
  const course = "React JS";  
  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>Course: {course}</p>  
    </div>  
  );  
};  
  
export default Student;
```

Footer.jsx

```
const Footer = () => {  
  return (  
    <footer>  
      <p>Student Portal 2026</p>  
    </footer>  
  );  
};
```

```
    </footer>
  );
};

export default Footer;
```

App.jsx

```
import Header from "../components/Header";
import Student from "../components/Student";
import Footer from "../components/Footer";

const App = () => {
  return (
    <>
      <Header />
      <Student />
      <Footer />
    </>
  );
};

export default App;
```

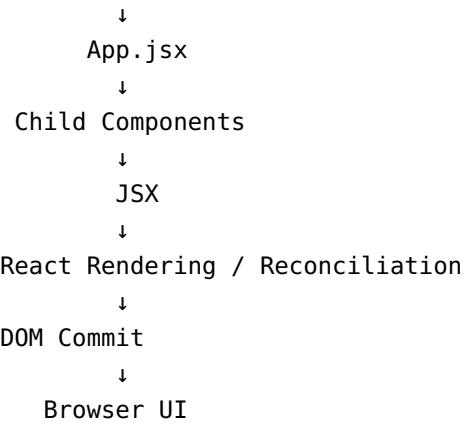
32. Complete React Application Flow

When we run:

```
npm run dev
```

Conceptually:

```
Browser loads index.html
↓
#root
↓
main.jsx initializes React
↓
createRoot()
↓
<App />
```



33. Important React Terms

Term	Meaning
React	JavaScript library for building UIs
Component	Reusable UI unit
JSX	HTML-like syntax used inside JavaScript
Props	Data passed from parent to child
State	Component data that can trigger rendering updates
Real DOM	Browser's actual DOM tree
Virtual DOM	Common term for React's in-memory UI representation
Reconciliation	Process of determining required UI updates
Render	React evaluates components to determine UI
Commit	React applies required changes to the DOM
App.jsx	Common root component filename
main.jsx	Common Vite React entry point

34. React Project Flow

```
index.html
  ↓
main.jsx
```

```
↓  
App.jsx  
↓  
Components  
↓  
JSX  
↓  
UI
```

35. Component Flow

```
Create Component  
↓  
Return JSX  
↓  
Export  
↓  
Import  
↓  
<Component />  
↓  
Display UI
```

36. React – 3 Main Areas

A. Create Phase

Project setup:

```
Project Setup  
↓  
Dependencies  
↓  
Files  
↓  
Folders  
↓  
Git
```

CRA

Older/common approach:

```
npx create-react-app intro
```

Vite

Modern setup:

```
npm create vite@latest introvite
```

B. Components

Components can be:

```
Single Component
  ↓
Multiple Components
  ↓
Component Composition
```

Example:

```
App
├─ Header
├─ Content
└─ Footer
```

C. Rendering / Updating

Important concepts:

```
Rendering
  ↓
Component Rendering
  ↓
State Update
```

↓
Reconciliation
↓
DOM Commit

Later topics include:

React Router DOM
↓
Navigation

37. Component Sharing

Components can be shared between files using:

Default Export

```
export default Header;
```

Import:

```
import Header from "../Header";
```

Named Export

```
export { Header };
```

Import:

```
import { Header } from "../Header";
```

38. Basic App.jsx Syntax

```
const App = () => {
```

```
    return (  
      <>  
        <div></div>  
      </>  
    );  
  };  
};
```

Meaning

```
const App  
  ↓  
Arrow Function  
  ↓  
return()  
  ↓  
JSX  
  ↓  
React UI
```

39. Important Interview Questions

What is React?

React is an open-source JavaScript library used to build interactive, component-based user interfaces.

Is React a library or framework?

React itself is primarily a **UI library**. Complete applications commonly use additional tools and libraries for routing, data fetching, state management, and other functionality.

What is a component?

A component is a reusable UI unit, commonly written as a JavaScript function that returns JSX.

Why are components important?

They improve:

- Reusability
- Maintainability
- Organization
- Readability

- Separation of concerns

What is JSX?

JSX is a syntax extension that allows HTML-like UI markup to be written inside JavaScript.

Can browsers directly execute JSX?

No. JSX normally needs to be transformed into JavaScript by development/build tooling.

What is Virtual DOM?

Virtual DOM is a common term for React's in-memory representation of UI used as part of its rendering and reconciliation process.

What is Real DOM?

The Real DOM is the browser's actual Document Object Model.

What is reconciliation?

Reconciliation is React's process of determining how a new render differs from the previous render and what updates are required.

What is App.jsx?

`App.jsx` is commonly used as the root application component in Vite React projects. The filename itself is a convention, not a React requirement.

What is main.jsx?

`main.jsx` is commonly the entry point that creates the React root and renders the root component.

Why should component names start with capital letters?

React uses capitalization to distinguish custom React components from built-in/host elements such as `div` and `button`.

40. Day 1 Golden Answer

React is an open-source JavaScript library used to build interactive, component-based user interfaces. React applications are divided into reusable components, and JSX is commonly used to describe the UI. Instead of manually updating individual DOM elements throughout the application, developers describe the UI based on data and state, and React manages the rendering process.

When props or state change, React renders the relevant components again and performs reconciliation to determine the necessary UI changes. Those changes are then committed to the browser's Real DOM. In a typical Vite React project, `main.jsx` initializes the React application, `App.jsx` commonly acts as the root application component, and additional components are imported and composed to build the complete interface.

41. DAY 1 – MUST REMEMBER

React Core Flow

```
React
  ↓
Components
  ↓
JSX
  ↓
Data / State
  ↓
Render
  ↓
Reconciliation
  ↓
Commit
  ↓
Real DOM
  ↓
Browser UI
```

Project Flow

```
index.html
  ↓
main.jsx
  ↓
App.jsx
  ↓
Components
  ↓
JSX
  ↓
UI
```

Rendering Flow

```
graph TD;
    A[User Action] --> B[Event];
    B --> C[State Update];
    C --> D[Render];
    D --> E[Reconciliation];
    E --> F[Commit];
    F --> G[Browser UI];
```

Component Flow

```
graph TD;
    A[Create] --> B[Return JSX];
    B --> C[Export];
    C --> D[Import];
    D --> E["<Component />"];
    E --> F[UI];
```